Esraa Ashraf Gamal

Task4

1-

```
module tesla_controller #(
    parameter MIN_DISTANCE = 7'd40
)(
    input  wire        clk,
    input  wire        rst,
    input  wire [7:0]  speed_limit,
    input  wire [7:0]  car_speed,
    input  wire [6:0]  leading_distance,
    output reg         unlock_doors,
    output reg         accelerate_car
);

    localparam STOP       = 2'b00;
    localparam ACCELERATE = 2'b01;
    localparam DECELERATE = 2'b10;

    reg [1:0] current_state, next_state;

    always @(posedge clk or posedge rst) begin
        if (rst)
            current_state <= STOP;
        else
            current_state <= next_state;
    end

    always @(*) begin
        case (current_state)

            STOP: begin
                if (leading_distance >= MIN_DISTANCE)
                    next_state = ACCELERATE;
                else
                    next_state = STOP;
            end

            ACCELERATE: begin
                if (leading_distance < MIN_DISTANCE)
                    next_state = DECELERATE;
```

```
            else if (car_speed >= speed_limit)
                next_state = DECELERATE;
            else
                next_state = ACCELERATE;
        end

        DECELERATE: begin
            if (car_speed == 8'd0)
                next_state = STOP;
            else if ((leading_distance >= MIN_DISTANCE) &&
                     (car_speed < speed_limit))
                next_state = ACCELERATE;
            else
                next_state = DECELERATE;
        end

        default: next_state = STOP;

    endcase
end

always @(*) begin
    case (current_state)

        STOP: begin
            unlock_doors   = 1'b1;
            accelerate_car = 1'b0;
        end

        ACCELERATE: begin
            unlock_doors   = 1'b0;
            accelerate_car = 1'b1;
        end

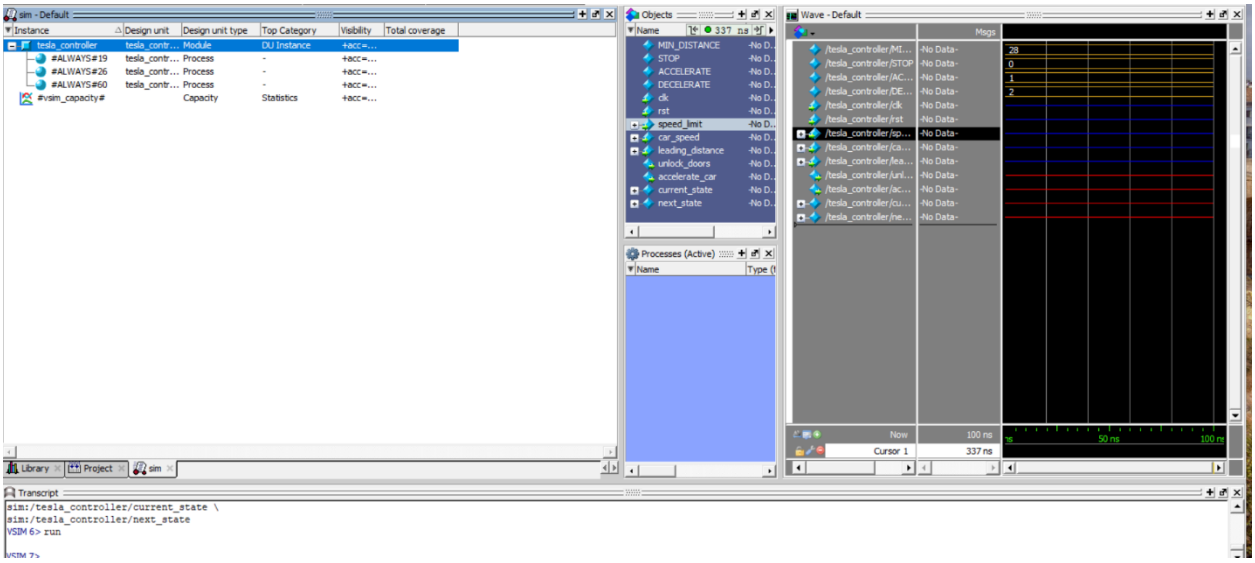
        DECELERATE: begin
            unlock_doors   = 1'b0;
            accelerate_car = 1'b0;
        end

        default: begin
            unlock_doors   = 1'b0;
            accelerate_car = 1'b0;
        end

    endcase
```

```
    end

endmodule
```



2-

```
module gray_counter_moore (
    input wire clk,
    input wire rst,
    output reg [1:0] y
);

    localparam A = 2'b00,
               B = 2'b01,
               C = 2'b10,
               D = 2'b11;

    reg [1:0] state, next_state;

    always @(posedge clk or posedge rst) begin
        if (rst)
            state <= A;
        else
            state <= next_state;
    end

    always @(*) begin
        case (state)
            A: next_state = B;
            B: next_state = C;
            C: next_state = D;
            D: next_state = A;
            default: next_state = A;
        endcase
    end

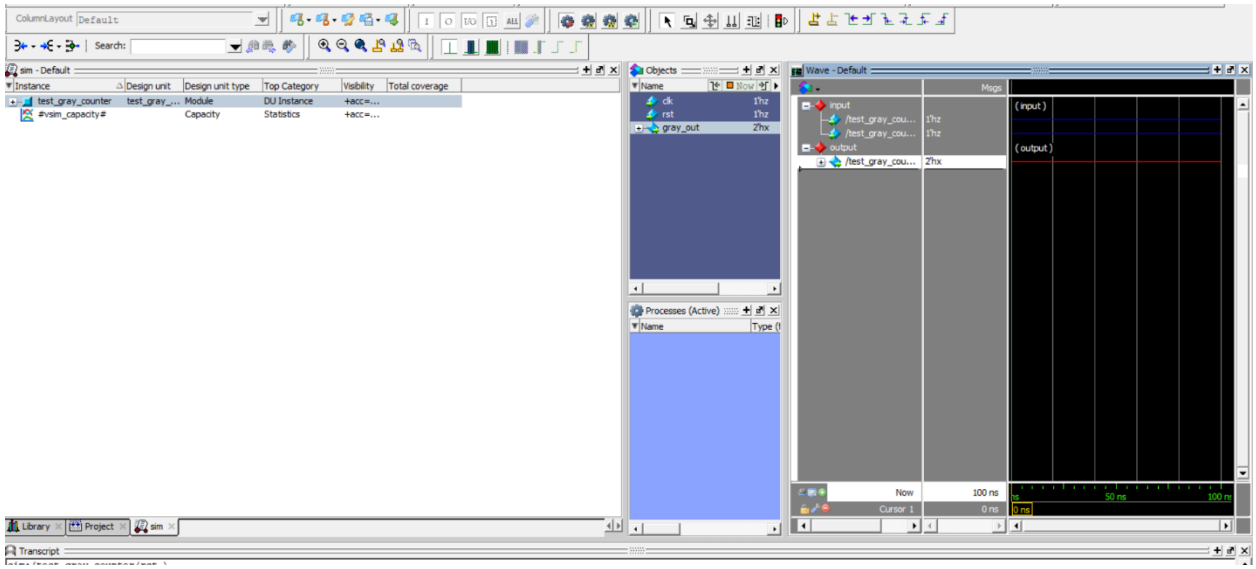
    always @(*) begin
        case (state)
            A: y = 2'b00;
            B: y = 2'b01;
            C: y = 2'b11;
            D: y = 2'b10;
            default: y = 2'b00;
        endcase
    end

endmodule
```

```
module test_gray_counter (
    input wire clk,
    input wire rst,
    output wire [1:0] gray_out
);

    gray_counter_moore u_gray_counter (
        .clk(clk),
        .rst(rst),
        .y(gray_out)
    );

endmodule
```



3-

```
module single_port_ram #(
    parameter MEM_WIDTH     = 16,
    parameter MEM_DEPTH     = 1024,
    parameter ADDR_SIZE     = 10,
    parameter ADDR_PIPELINE = 0,
    parameter DOUT_PIPELINE = 1,
    parameter PARITY_ENABLE = 1
)(
    input  wire                     clk,
    input  wire                     rst,
    input  wire                     blk_select,
    input  wire                     wr_en,
    input  wire                     rd_en,
    input  wire                     addr_en,
    input  wire                     dout_en,
    input  wire [ADDR_SIZE-1:0]     addr,
    input  wire [MEM_WIDTH-1:0]     din,
    output reg  [MEM_WIDTH-1:0]     dout,
    output reg                      parity_out
);

    reg [MEM_WIDTH-1:0] mem [0:MEM_DEPTH-1];

    reg [ADDR_SIZE-1:0] addr_reg;
    wire [ADDR_SIZE-1:0] addr_int;

    generate
        if (ADDR_PIPELINE) begin
            always @(posedge clk) begin
                if (rst)
                    addr_reg <= 0;
                else if (addr_en)
                    addr_reg <= addr;
            end
            assign addr_int = addr_reg;
        end else begin
            assign addr_int = addr;
        end
    endgenerate

    reg [MEM_WIDTH-1:0] dout_reg;
```

```
    wire [MEM_WIDTH-1:0] dout_int;

    always @(posedge clk) begin
        if (blk_select && wr_en)
            mem[addr_int] <= din;
    end

    always @(posedge clk) begin
        if (blk_select && rd_en)
            dout_reg <= mem[addr_int];
    end

    generate
        if (DOUT_PIPELINE) begin
            always @(posedge clk) begin
                if (rst)
                    dout <= 0;
                else if (dout_en)
                    dout <= dout_reg;
            end
            assign dout_int = dout;
        end else begin
            always @(*) begin
                dout = dout_reg;
            end
            assign dout_int = dout;
        end
    endgenerate

    generate
        if (PARITY_ENABLE) begin
            always @(posedge clk) begin
                if (rst)
                    parity_out <= 0;
                else if (blk_select && rd_en)
                    parity_out <= ^dout_int;
            end
        end else begin
            always @(*) begin
                parity_out = 1'b0;
            end
        end
    endgenerate

endmodule
```

```
module single_port_ram_tb;

    parameter MEM_WIDTH     = 16;
    parameter MEM_DEPTH     = 1024;
    parameter ADDR_SIZE     = 10;

    reg clk;
    reg rst;
    reg blk_select;
    reg wr_en;
    reg rd_en;
    reg addr_en;
    reg dout_en;
    reg [ADDR_SIZE-1:0] addr;
    reg [MEM_WIDTH-1:0] din;

    wire [MEM_WIDTH-1:0] dout;
    wire parity_out;

    single_port_ram #(
        .MEM_WIDTH(MEM_WIDTH),
        .MEM_DEPTH(MEM_DEPTH),
        .ADDR_SIZE(ADDR_SIZE),
        .ADDR_PIPELINE(1),
        .DOUT_PIPELINE(1),
        .PARITY_ENABLE(1)
    ) dut (
        .clk(clk),
        .rst(rst),
        .blk_select(blk_select),
        .wr_en(wr_en),
        .rd_en(rd_en),
        .addr_en(addr_en),
        .dout_en(dout_en),
        .addr(addr),
        .din(din),
        .dout(dout),
        .parity_out(parity_out)
    );

    always #5 clk = ~clk;

    initial begin
```

```
        clk = 0;
        rst = 1;
        blk_select = 0;
        wr_en = 0;
        rd_en = 0;
        addr_en = 0;
        dout_en = 0;
        addr = 10'b0000000000;
        din  = 16'b0000000000000000;

        #20;
        rst = 0;
        blk_select = 1;

        addr_en = 1;
        addr = 10'b0000001010;
        din  = 16'b1010010110100101;
        #10;

        wr_en = 1;
        #10;

        wr_en = 0;
        addr_en = 0;
        #10;

        addr_en = 1;
        addr = 10'b0000001010;
        #10;

        rd_en = 1;
        dout_en = 1;
        #30;

        rd_en = 0;
        addr_en = 0;
        #10;

        addr_en = 1;
        addr = 10'b0000010100;
        din  = 16'b0101010110101010;
        #10;

        wr_en = 1;
        #10;
```

```
        wr_en = 0;
        addr_en = 0;
        #10;

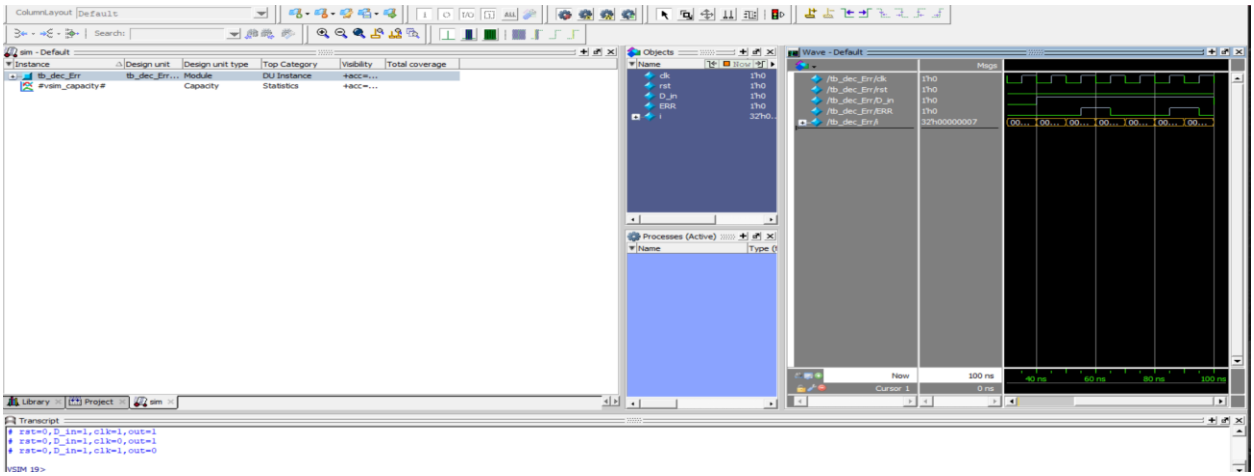
        addr_en = 1;
        addr = 10'b0000010100;
        #10;

        rd_en = 1;
        #30;

        rd_en = 0;
        addr_en = 0;

        #40;
        $stop;
    end

endmodule
```



```
sim:/tb_dec_Err/i
VSIM 18> run
# rst=1,D_in=0,clk=0,out=0
# rst=1,D_in=0,clk=1,out=0
# rst=0,D_in=0,clk=0,out=0
# rst=0,D_in=0,clk=1,out=0
# rst=0,D_in=0,clk=0,out=0
# rst=0,D_in=0,clk=1,out=0
# rst=0,D_in=0,clk=0,out=0
# rst=0,D_in=0,clk=1,out=0
# rst=0,D_in=1,clk=0,out=0
# rst=0,D_in=1,clk=1,out=0
# rst=0,D_in=1,clk=0,out=0
# rst=0,D_in=1,clk=1,out=1
# rst=0,D_in=1,clk=0,out=1
# rst=0,D_in=1,clk=1,out=0
# rst=0,D_in=1,clk=0,out=0
# rst=0,D_in=1,clk=1,out=0
# rst=0,D_in=1,clk=0,out=0
# rst=0,D_in=1,clk=1,out=1
# rst=0,D_in=1,clk=0,out=1
# rst=0,D_in=1,clk=1,out=0
```

4-

```
 module single_port_ram #(

    parameter MEM_WIDTH     = 16,
    parameter MEM_DEPTH     = 1024,
    parameter ADDR_SIZE     = 10,
    parameter ADDR_PIPELINE = 0,
    parameter DOUT_PIPELINE = 1,
    parameter PARITY_ENABLE = 1
)(
    input  wire                     clk,
    input  wire                     rst,
    input  wire                     blk_select,
    input  wire                     wr_en,
    input  wire                     rd_en,
    input  wire                     addr_en,
    input  wire                     dout_en,
    input  wire [ADDR_SIZE-1:0]     addr,
    input  wire [MEM_WIDTH-1:0]     din,
    output reg  [MEM_WIDTH-1:0]     dout,
    output reg                      parity_out
);

    reg [MEM_WIDTH-1:0] mem [0:MEM_DEPTH-1];

    reg [ADDR_SIZE-1:0] addr_reg;
    wire [ADDR_SIZE-1:0] addr_int;

    generate
        if (ADDR_PIPELINE) begin
            always @(posedge clk) begin
                if (rst)
                    addr_reg <= 0;
                else if (addr_en)
                    addr_reg <= addr;
            end
            assign addr_int = addr_reg;
        end else begin
            assign addr_int = addr;
        end
    endgenerate

    reg [MEM_WIDTH-1:0] dout_reg;
    wire [MEM_WIDTH-1:0] dout_int;
```

```
    always @(posedge clk) begin
        if (blk_select && wr_en)
            mem[addr_int] <= din;
    end

    always @(posedge clk) begin
        if (blk_select && rd_en)
            dout_reg <= mem[addr_int];
    end

    generate
        if (DOUT_PIPELINE) begin
            always @(posedge clk) begin
                if (rst)
                    dout <= 0;
                else if (dout_en)
                    dout <= dout_reg;
            end
            assign dout_int = dout;
        end else begin
            always @(*) begin
                dout = dout_reg;
            end
            assign dout_int = dout;
        end
    endgenerate

    generate
        if (PARITY_ENABLE) begin
            always @(posedge clk) begin
                if (rst)
                    parity_out <= 0;
                else if (blk_select && rd_en)
                    parity_out <= ^dout_int;
            end
        end else begin
            always @(*) begin
                parity_out = 1'b0;
            end
        end
    endgenerate

endmodule
```

```
module single_port_ram_tb;

    parameter MEM_WIDTH     = 16;
    parameter MEM_DEPTH     = 1024;
    parameter ADDR_SIZE     = 10;

    reg clk;
    reg rst;
    reg blk_select;
    reg wr_en;
    reg rd_en;
    reg addr_en;
    reg dout_en;
    reg [ADDR_SIZE-1:0] addr;
    reg [MEM_WIDTH-1:0] din;

    wire [MEM_WIDTH-1:0] dout;
    wire parity_out;

    single_port_ram #(
        .MEM_WIDTH(MEM_WIDTH),
        .MEM_DEPTH(MEM_DEPTH),
        .ADDR_SIZE(ADDR_SIZE),
        .ADDR_PIPELINE(1),
        .DOUT_PIPELINE(1),
        .PARITY_ENABLE(1)
    ) dut (
        .clk(clk),
        .rst(rst),
        .blk_select(blk_select),
        .wr_en(wr_en),
        .rd_en(rd_en),
        .addr_en(addr_en),
        .dout_en(dout_en),
        .addr(addr),
        .din(din),
        .dout(dout),
        .parity_out(parity_out)
    );
```

```
    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rst = 1;
        blk_select = 0;
        wr_en = 0;
        rd_en = 0;
        addr_en = 0;
        dout_en = 0;
        addr = 10'b0000000000;
        din  = 16'b0000000000000000;

        #20;
        rst = 0;
        blk_select = 1;

        addr_en = 1;
        addr = 10'b0000001010;
        din  = 16'b1010010110100101;
        #10;

        wr_en = 1;
        #10;

        wr_en = 0;
        addr_en = 0;
        #10;

        addr_en = 1;
        addr = 10'b0000001010;
        #10;

        rd_en = 1;
        dout_en = 1;
        #30;

        rd_en = 0;
        addr_en = 0;
        #10;

        addr_en = 1;
        addr = 10'b0000010100;
        din  = 16'b0101010110101010;
        #10;
```

```
        wr_en = 1;
        #10;

        wr_en = 0;
        addr_en = 0;
        #10;

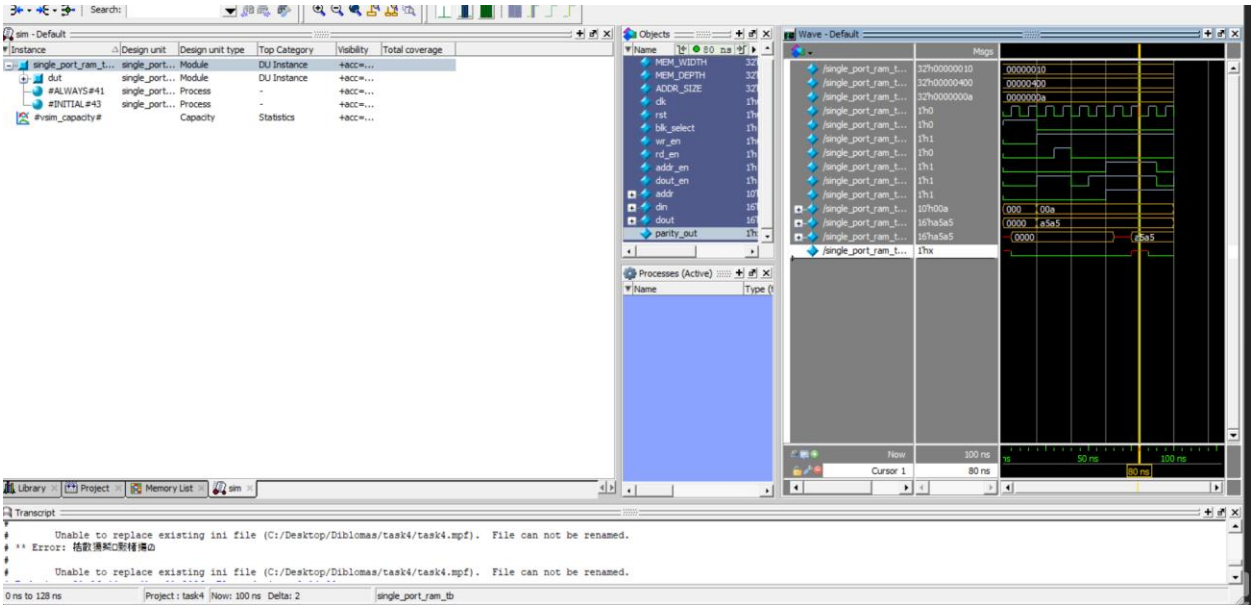
        addr_en = 1;
        addr = 10'b0000010100;
        #10;

        rd_en = 1;
        #30;

        rd_en = 0;
        addr_en = 0;

        #40;
        $stop;
    end

endmodule
```


sim - Default
Instance
Design unit
Design unit type
Top Category
Visibility
Total coverage
single_port_ram_t...
single_port...
Module
DU Instance
+acc=...
dut
single_port...
Module
DU Instance
+acc=...
#ALWAYS#41
single_port...
Process
-
+acc=...
#INITIAL#43
single_port...
Process
-
+acc=...
#vsim_capacity#
Capacity
Statistics
+acc=...
Objects
Name
MEM_WIDTH
MEM_DEPTH
ADDR_SIZE
clk
rst
blk_select
wr_en
rd_en
addr_en
dout_en
addr
din
dout
parity_out
Processes (Active)
Name
Wave - Default
Msgs
/single_port_ram_t...
32'h00000010
32'h00000400
32'h0000000a
1'h0
1'h1
10'h00a
16'ha5a5
1'hx
Now
100 ns
Cursor 1
80 ns
50 ns
100 ns
Library
Project
Memory List
sim
Transcript
Unable to replace existing ini file (C:/Desktop/Diblomas/task4/task4.mpf). File can not be renamed.
** Error: 格数据和已取得值の
#
Unable to replace existing ini file (C:/Desktop/Diblomas/task4/task4.mpf). File can not be renamed.
0 ns to 128 ns
Project : task4
Now: 100 ns Delta: 2
single_port_ram_tb

5-

```
  module FIFO (din_a,wen_a,ren_b,clk_a,clk_b,rst,dout_b,full,empty);


parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH =512;
parameter ADDR_WIDTH = $clog2 (FIFO_DEPTH);

input [FIFO_WIDTH-1 : 0] din_a;
input wen_a ,ren_b , clk_a ,clk_b ,rst;
output reg[FIFO_WIDTH-1 :0] dout_b ;
output full , empty ;

reg [FIFO_WIDTH-1 : 0] mem [FIFO_DEPTH-1:0];

reg [ADDR_WIDTH:0] wrp_b,rdp_b ;
wire [ADDR_WIDTH:0] wrp_g,rdp_g ;
reg [ADDR_WIDTH:0] wrp_g_sync1 , rdp_g_sync1 ;
reg [ADDR_WIDTH:0] wrp_g_sync2 , rdp_g_sync2 ;

always @(posedge clk_a) begin
    if(rst)begin
        wrp_b<=0;
    end
 else
      if (wen_a && !full) begin
    mem[wrp_b[ADDR_WIDTH-1:0]] <= din_a;
    wrp_b <= wrp_b+1;
 end
end

assign wrp_g=wrp_b^(wrp_b>>1);

always @(posedge clk_a) begin
    if(rst) begin
        rdp_g_sync1 <= 0;
         rdp_g_sync2 <= 0;
    end
 else  begin
    rdp_g_sync1 <= rdp_g ;
    rdp_g_sync2 <= rdp_g_sync1 ;
 end
```

```
end
assign full =(wrp_g=={~rdp_g_sync2[ADDR_WIDTH:ADDR_WIDTH-1] ,
rdp_g_sync2[ADDR_WIDTH-2:0]});

always @(posedge clk_b) begin
    if(rst)begin
        rdp_b <=0;
        dout_b <= 0;
    end
  else if (ren_b && !empty ) begin
     dout_b <= mem[rdp_b[ADDR_WIDTH-1 :0]];
     rdp_b <= rdp_b + 1 ;
 end
end
assign rdp_g=rdp_b^(rdp_b>>1);

always @(posedge clk_b) begin
    if(rst) begin
        wrp_g_sync1<=0;
        wrp_g_sync2<=0;
    end
  else begin
    wrp_g_sync1 <= wrp_g;
    wrp_g_sync2 <= wrp_g_sync1;
  end
end
assign empty=(rdp_g == wrp_g_sync2);

endmodule
```

```
module FIFO_tb;

    parameter FIFO_WIDTH = 16;
    parameter FIFO_DEPTH = 8;
    parameter ADDR_WIDTH = $clog2(FIFO_DEPTH);

    reg [FIFO_WIDTH-1:0] din_a;
    reg wen_a, ren_b, clk_a, clk_b, rst;

    wire [FIFO_WIDTH-1:0] dout_b;
    wire full, empty;

    FIFO #(FIFO_WIDTH, FIFO_DEPTH) uut (
        .din_a(din_a),
        .wen_a(wen_a),
        .ren_b(ren_b),
        .clk_a(clk_a),
        .clk_b(clk_b),
        .rst(rst),
        .dout_b(dout_b),
        .full(full),
        .empty(empty)
    );

    initial begin
        clk_a = 0;
        forever #4 clk_a = ~clk_a;
    end

    initial begin
        clk_b = 0;
        forever #10 clk_b = ~clk_b;
    end

    initial begin
        rst = 1;
        wen_a = 0;
        ren_b = 0;
        din_a = 0;

        repeat(5) @(posedge clk_a);
        repeat(5) @(posedge clk_b);
        rst = 0;

        $display("Starting Write Operation...");
```

```
        repeat (10) begin
            @(posedge clk_a);
            if (!full) begin
                wen_a = 1;
                din_a = din_a + 1;
            end else begin
                wen_a = 0;
                $display("FIFO FULL at time %t", $time);
            end
            @(posedge clk_a);
            wen_a = 0;
        end

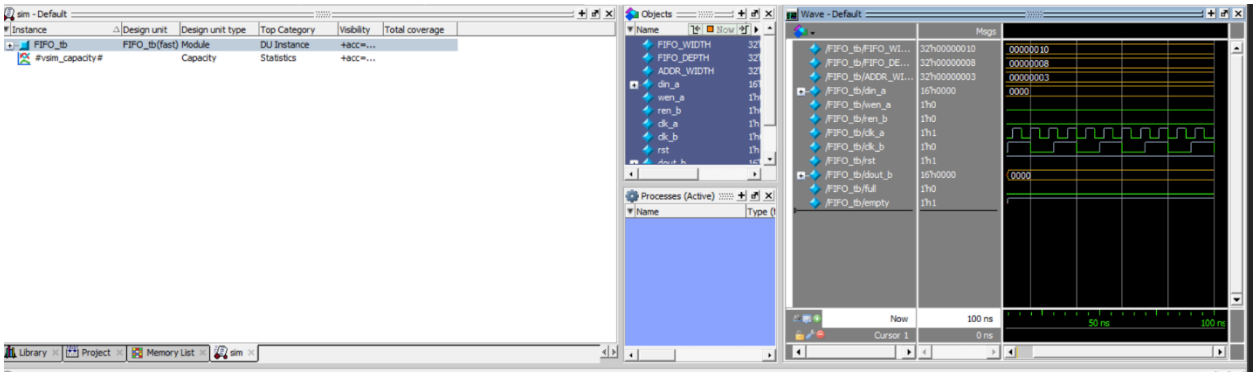
        repeat(5) @(posedge clk_a);

        $display("Starting Read Operation...");

        repeat (10) begin
            @(posedge clk_b);
            if (!empty) begin
                ren_b = 1;
                @(posedge clk_b);
                ren_b = 0;
                $display("Read Data: %d at time %t", dout_b, $time);
            end else begin
                ren_b = 0;
                $display("FIFO EMPTY at time %t", $time);
            end
        end

        $display("Finished!");
        $stop;
    end

endmodule
```




```
# ** Error: 格散遭契□敷補痛□
#
#       Unable to replace existing ini file (C:/Desktop/Diblomas/task4/task4.mpf).  File can not be renamed.
# End time: 21:59:22 on Mar 01,2026, Elapsed time: 0:10:03
# Errors: 24, Warnings: 8
# vsim -voptargs="+acc" work.FIFO_tb
# Start time: 21:59:22 on Mar 01,2026
# ** Note: (vsim-3812) Design is being optimized...
# Loading work.FIFO_tb(fast)
# Loading work.FIFO(fast)
add wave -position insertpoint  \
sim:/FIFO_tb/FIFO_WIDTH \
sim:/FIFO_tb/FIFO_DEPTH \
```

6-

```
module DSP(A, B, C, D, clk, rst_n, P);
parameter OPERATION = "ADD";
input [17:0] A, B, D;
input [47:0] C;
input clk, rst_n;
output reg [47:0] P;

reg [17:0] A1, A2, B_reg, D_reg;
reg [47:0] C_reg, adder_out, mult_out;

// Stage 1: Input registers
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        A1 <= 0;
        B_reg <= 0;
        D_reg <= 0;
        C_reg <= 0;
    end else begin
        A1 <= A;
        B_reg <= B;
        D_reg <= D;
        C_reg <= C;
    end
end

// Stage 2: Adder and A delay
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        A2 <= 0;
        adder_out <= 0;
    end else begin
        A2 <= A1;
        if (OPERATION == "ADD")
            adder_out <= D_reg + B_reg;
        else
            adder_out <= D_reg - B_reg;
    end
end

// Stage 3: Multiplier
always @(posedge clk or negedge rst_n) begin
```

```
    if (!rst_n)
        mult_out <= 0;
    else
        mult_out <= A2 * adder_out;
end

// Stage 4: Accumulator
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        P <= 0;
    else begin
        if (OPERATION == "ADD")
            P <= mult_out + C_reg;
        else
            P <= mult_out - C_reg;
    end
end

endmodule
```

```
module DSP_tb;

    reg [17:0] A, B, D;
    reg [47:0] C;
    reg clk, rst_n;
    wire [47:0] P;

    parameter OPERATION = "ADD";

    DSP #(.OPERATION(OPERATION)) dut (
        .A(A),
        .B(B),
        .C(C),
        .D(D),
        .clk(clk),
        .rst_n(rst_n),
        .P(P)
    );

    reg [47:0] expected;
    integer i;

    initial begin
        clk = 0;
```

```
        forever #5 clk = ~clk;
    end

    initial begin
        rst_n = 0;
        A = 0; B = 0; C = 0; D = 0;
        repeat(5) @(posedge clk);
        rst_n = 1;

        for (i = 0; i < 10; i = i + 1) begin
            @(posedge clk);

            A = $random;
            B = $random;
            D = $random;
            C = $random;

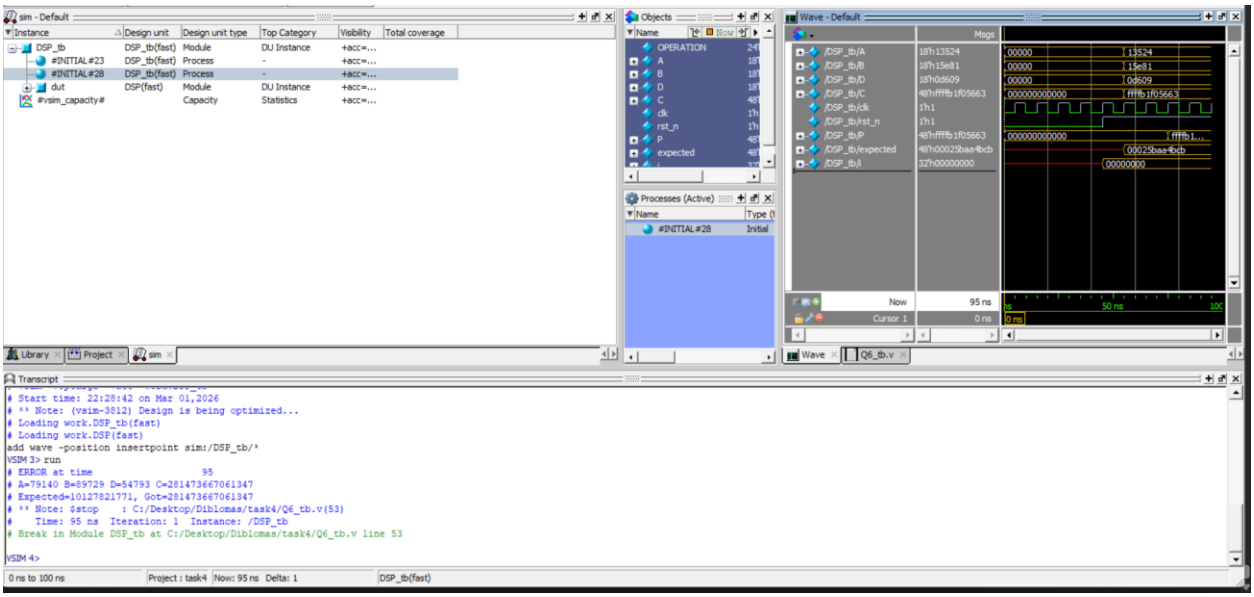
            if (OPERATION == "ADD")
                expected = (A * (D + B)) + C;
            else
                expected = (A * (D - B)) - C;

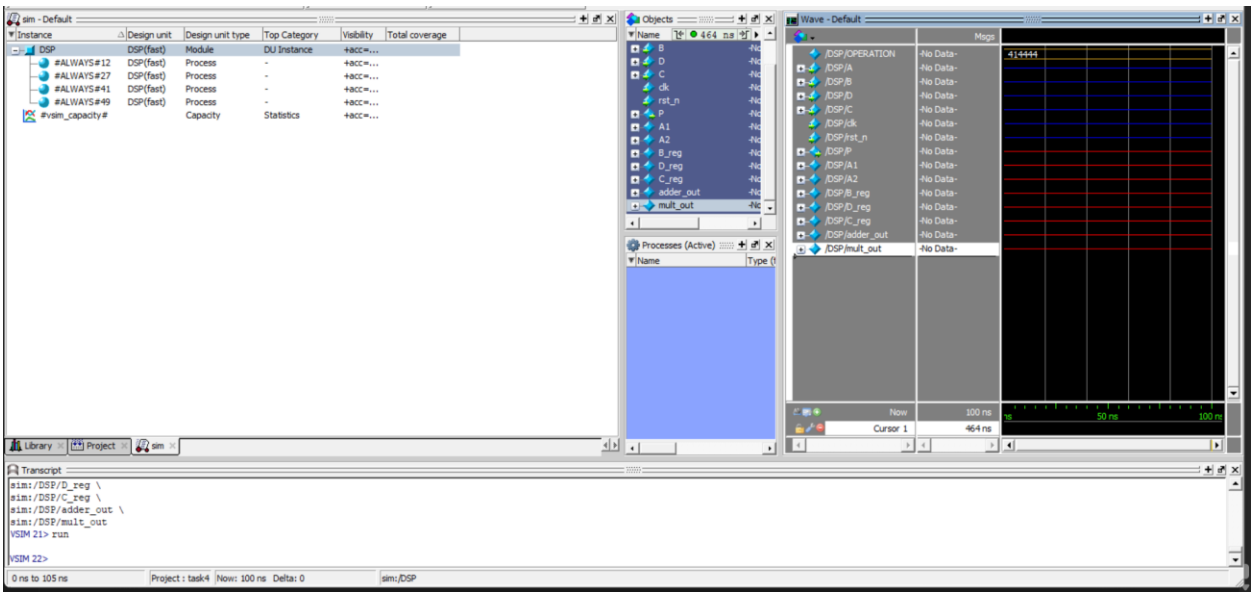
            repeat(4) @(posedge clk);

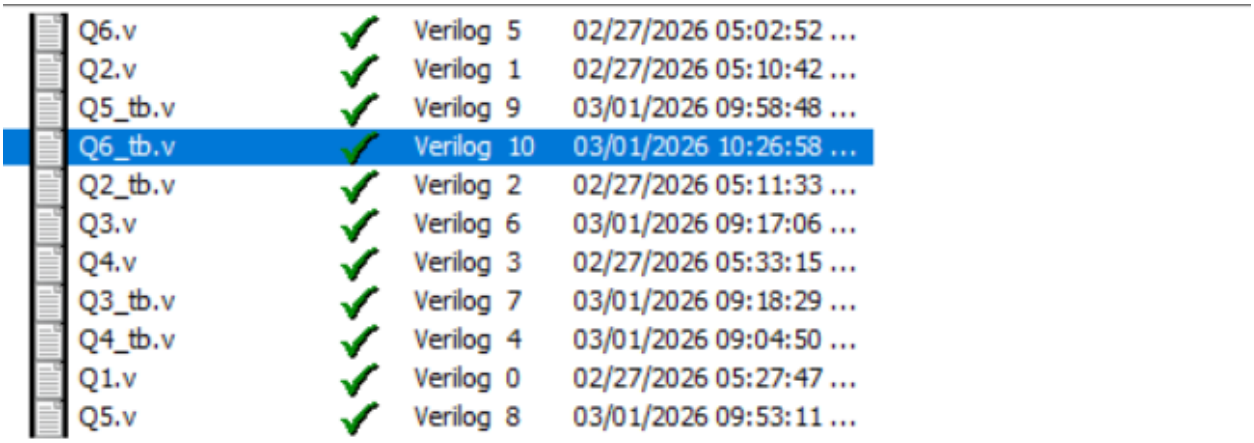
            if (P !== expected) begin
                $display("ERROR at time %t", $time);
                $display("A=%0d B=%0d D=%0d C=%0d", A, B, D, C);
                $display("Expected=%0d, Got=%0d", expected, P);
                $stop;
            end else begin
                $display("PASS at time %t | Result=%0d", $time, P);
            end
        end

        $display("All tests PASSED.");
        $stop;
    end

endmodule
```


sim - Default
Objects
Wave - Default
Processes (Active)
Transcript
Start time: 22:28:42 on Mar 01,2026
** Note: (vsim-3812) Design is being optimized...
Loading work.DSP_tb(fast)
Loading work.DSP(fast)
add wave -position insertpoint sim:/DSP_tb/*
VSIM 3> run
ERROR at time 95
A=79140 B=89729 D=54793 C=281473667061347
Expected=10127821771, Got=281473667061347
** Note: $stop : C:/Desktop/Diblomas/task4/Q6_tb.v(53)
Time: 95 ns Iteration: 1 Instance: /DSP_tb
Break in Module DSP_tb at C:/Desktop/Diblomas/task4/Q6_tb.v line 53
VSIM 4>


sim - Default
Objects
Wave - Default
Processes (Active)
Transcript
sim:/DSP/D_reg \
sim:/DSP/C_reg \
sim:/DSP/adder_out \
sim:/DSP/mult_out
VSIM 21> run
VSIM 22>


Q6.v Verilog 5 02/27/2026 05:02:52 ...
Q2.v Verilog 1 02/27/2026 05:10:42 ...
Q5_tb.v Verilog 9 03/01/2026 09:58:48 ...
Q6_tb.v Verilog 10 03/01/2026 10:26:58 ...
Q2_tb.v Verilog 2 02/27/2026 05:11:33 ...
Q3.v Verilog 6 03/01/2026 09:17:06 ...
Q4.v Verilog 3 02/27/2026 05:33:15 ...
Q3_tb.v Verilog 7 03/01/2026 09:18:29 ...
Q4_tb.v Verilog 4 03/01/2026 09:04:50 ...
Q1.v Verilog 0 02/27/2026 05:27:47 ...
Q5.v Verilog 8 03/01/2026 09:53:11 ...